TAB B — UI Improvements

B.1 The current access-management application

The `cwms-access-management` repository is an Nx monorepo (pnpm, Node 24, TypeScript 5) containing a Fastify authorization proxy, a Fastify management API, a Commander/Ink CLI, and a React 18 management UI built with Vite, Tailwind, TanStack Query, react-router, and zustand. The UI has five pages — Home, Login, Users, Roles, Policies — and it is, in its entirety, **read-only**. The Users page is a table of username, email, full name, and an Active badge with client-side search; the `User` type in the UI's API service omits the `offices` and `roles` fields that the management API actually returns, so district scope is invisible. The Roles page lists role names and IDs; every description renders as "–" because the flat `GET /roles` in CDA returns only distinct group identifiers with no office. The Policies page lists Rego files read from OPA's `/v1/policies` endpoint and renders them with a syntax highlighter. The management API answers `POST` and `DELETE` on users and roles with HTTP 501. A Keycloak admin service exists in the code and is not wired to anything. In short: the previous effort delivered the frame and the read paths; the write paths, the district scoping, and the policy editing that TE1 describes do not exist yet.

Three further shortcomings shape the design, all traced to specific files:

1. **District scope has no representation in the UI or the management API.** CDA's own user endpoints (`GET /users?office=…`, `POST/DELETE /user/{user-name}/roles/{office-id}`) are office-scoped and already enforce the `CWMS User Admins` role; the management API calls `GET /users?include-roles=true&page-size=1000` once and flattens roles across offices. Every TE1 requirement that says "for their district" depends on restoring that scope end to end.
2. **Policy data is not data.** Offices and regions are a hard-coded map in `policies/helpers/offices.rego` (with a comment promising to load them "in a future sprint"); embargo hours are parsed from a naming convention on time-series-group identifiers; per-persona constraints are computed in the proxy's TypeScript. A Policy tab that lets a district administrator "create policy and perform at least basic validation" cannot be built against files that are mounted into the OPA container and require a container restart to change.
3. **There is no CDA endpoint for the security time-series groups.** The schema has them — `at_sec_ts_groups`, `at_sec_ts_group_masks`, `at_sec_allow`, and the `cwms_sec` procedures that manage them — and the `/timeseries/group` endpoints look like the right thing but are the data-categorization groups in `at_ts_group`. "Assign a time series to a policy" therefore begins in Java, not in React.

The shortcomings are ordinary for a first increment, and they define the order of work.

B.2 Technical Exhibit 1, requirement by requirement

Table B-1. Gap analysis against TE1. *Exists* = works today; *Partial* = data path exists, UI does not; *Missing* = requires new CDA endpoint(s) or a source-of-truth decision.

TE1 requirement	State	Where the change lands	Increment
Users tab			
View all users with roles/permissions for their district	Partial	UI `UsersPage`, API `getUsers(office)` → CDA `GET /users?office=&include-roles=true`	U1
Assign roles to users for a specific district	Partial	UI assign control → mgmt API (new write route) → CDA `POST /user/{u}/roles/{office}` (exists)	U1
Show new users that do not yet have privileges	Missing	CDA `GET /users?unassigned=true&office=` (new param), UI "Unassigned" filter	U2 (§B.4)
Organize users by assigned roles	Partial	UI grouping on the roles field the API already returns	U1
Cannot assign roles for districts they lack permission on	Partial	CDA enforces via `is_user_admin(office)`; UI limits the office selector to the admin's offices from `/user/profile`	U1
Roles tab			
Create roles specific to a district's data	Missing	CDA endpoint wrapping `cwms_sec.create_user_group` / `delete_user_group` (office-scoped); UI create form	R1
Assign policies to roles	Missing	Policy = time-series-group grant (`assign_ts_group_user_group`) — see Policy tab; UI role detail page lists grants	R2
Sort/filter roles by assigned policies	Missing	Follows R2 data	R2
View/edit/filter roles for a district	Partial	`GET /roles` gains office scope (from `av_sec_user_groups`); UI filter	R1
Cannot edit roles for districts they lack permission on	Partial	CDA role check; UI scope	R1
Policy tab			
View policies associated with district data	Partial	Today: Rego viewer. After P1: list of security time-series groups with masks, grants, embargo for the district	P1
Assign a time series to a policy	Missing	CDA `/auth/ts-groups/{id}/masks` (new) → `assign_ts_masks_to_ts_group`; UI selector	P1 (§B.3)
Use time-series groups (or manage within the authorization system); restrictive or allowed	Missing	Security groups *are* the mechanism; allowed vs restrictive expressed as read/write/none grant per user group; UI toggle	P1
Assign by time-series parameters (LOC/parameter/version)	Missing	Mask pattern over the six-part identifier; UI mask builder with catalog preview	P1 (§B.3)

TE1 requirement	State	Where the change lands	Increment
Assign embargo to a time series	Missing	Embargo column/table decision (Tab A, ADR); CDA field; UI hours picker	P1 (§B.3)
Create policy and perform basic validation	Missing	CDA validation (mask syntax, office ownership, group-name uniqueness) + UI inline validation	P1
Cross-cutting			
Bulk manipulation of permissions (PWS Task 2)	Missing	Multi-select on Users and Policy tabs; batched CDA calls with per-row result	U3
Section 508 (Q&A 49)	Partial	Keyboard paths, labels, focus order, error messaging; Storybook stories per component	every increment

Increment order across the 180-day Task 2 period: **U1 → P1 → U2 → R1 → R2 → U3**, with documentation and training materials in the final increment. The PWS minimum successful product — an interface to manage access control and the access-control method working for time-series data — is reached at the end of P1.

B.3 One requirement in detail: assign a time series to a policy, scoped by location, parameter, and version, with an embargo

This is the requirement that touches every layer, so it is the one we take all the way down.

What a district administrator is doing. SWT's water manager wants raw stage data at a set of gauges to be readable by the district's cooperators only after 72 hours, while the district's own operators see it immediately. In CWMS terms: a security time-series group scoped to `*.Stage.Inst.*.0.Raw*` at those locations, granted *read* to the `external_cooperator` user group with a 72-hour embargo, and *read/write* to `dam_operator` with none.

Data model. We use the schema's own model rather than inventing a parallel one, because the database, VPD, and CDA's `UserDao` already understand it:

- `at_sec_ts_groups(db_office_code, ts_group_code, ts_group_id, description)` — the policy, owned by an office.
- `at_sec_ts_group_masks(db_office_code, ts_group_code, ts_group_mask)` — one or more LIKE patterns over the six-part time-series identifier `Location.Parameter.ParameterType.Interval.Duration.Version`; the location/parameter/version scoping TE1 asks for is a mask with wildcards in the other positions, normalized by `cwms_util` (`*` and `?` accepted).
- `at_sec_allow(db_office_code, ts_group_code, user_group_code, privilege_bit)` — the grant: read (2), write (4), or none.
- `at_sec_ts_group_embargo(ts_group_code, embargo_hours)` — reinstated from the previous effort's `001-ts-group-embargo.sql`, replacing the name-suffix convention (Tab A, ADR). `av_sec_ts_group_mask` and `av_sec_ts_privileges` are extended to carry the hours so that one view answers "what may this user see, and from when".

Policy evaluation. The proxy already sends OPA `{user:{offices, roles, ts_privileges[]}, resource, action, context}`; `ts_privileges` gains `embargo_hours` from the view instead of from a regex, and `helpers/time_rules.rego` — which already

contains `get_ts_group_embargo_hours` — reads it. The decision returned to CDA carries `ts_group_embargo` in the constraints block; `AuthorizationFilterHelper.getTsGroupEmbargoFilter` (present in PR #1461, never called) is wired into `TimeSeriesDaoImpl.getRequestedTimeSeries` beside the office filter, producing `date_time < now - embargo_hours` for matching identifiers. Embargoed rows are absent, not zeroed; the response's paging cursor is computed after the filter so that clients see a consistent window. A Rego unit test file per persona asserts the allow, the deny, and the embargo boundary at ±1 second.

CDA endpoints (new, `api/auth/`, role `CWMS User Admins`).

Method and path	Backed by	Purpose
`GET /auth/ts-groups?office=`	`av_sec_ts_group_mask`, `av_sec_ts_privileges`	List policies for a district with masks, grants, embargo
`POST /auth/ts-groups`	`cwms_sec.create_ts_group`	Create a policy (validates name uniqueness within office)
`PATCH /auth/ts-groups/{id}`	`change_ts_group_desc`, embargo table	Edit description or embargo hours
`PUT /auth/ts-groups/{id}/masks`	`assign_ts_masks_to_ts_group` (add/remove lists)	Set the location/parameter/version patterns
`PUT /auth/ts-groups/{id}/user-groups/{ug}`	`assign_ts_group_user_group` with `read`, `write`, or `none`	Grant or revoke
`GET /auth/ts-groups/{id}/preview?office=`	`GET /catalog/TIMESERIES` with `like=`	Show which identifiers a mask currently matches

Each is a jOOQ call into `cwms_sec` in the style of the existing `CWMS_TS_PACKAGE` bindings, with DTOs under `data/dto/auth/` (reusing `TsGroupPrivilege` from PR #1461), OpenAPI annotations so the static-analysis test passes, and an `*IT` test class in the `DataApiTestIT` pattern that creates a group as a `CWMS User Admins` user, assigns masks and a grant, then reads time series as an `external_cooperator` and asserts that rows inside the embargo window are absent and rows outside it are present.

Management API. A `routes/ts-groups.ts` that forwards the six calls with zod validation (`middleware/validation.ts`), and the office-scoped `getUsers`/`getRoles` changes from increment U1. The management API stops flattening roles across offices.

React UI — the Policy tab flow.

1. *Office selector* limited to the offices where `/user/profile` says the administrator holds `CWMS User Admins`. Everything on the page is scoped to it.
2. *Policy list* (replaces the Rego viewer as the default view; the viewer moves to an "Advanced" panel): name, description, mask count, grants, embargo, last modified.
3. *Create / edit policy* form: name, description, embargo hours (0 = none) — inline validation from the CDA response.

4. *Mask builder*: three fields — Location (with type-ahead from `/locations?office=`), Parameter (from `/parameters`), Version (free text) — and a generated pattern shown as text (`*.Stage.Inst.*.0.Raw*`) that an advanced user can edit directly. A **preview** panel calls the preview endpoint and lists the identifiers the pattern currently matches, with a count, so that an administrator sees the effect before saving. Multiple masks per policy.
5. *Grants*: a table of the district's user groups with a three-state control (none / read / read+write) per row; changes are batched into one `PUT` per changed row with per-row success or error shown in place.
6. *Bulk assignment* (increment U3): select several policies or several time series from the catalog and apply a grant or an embargo in one action, with a result table.

Components are built with Storybook stories and React Testing Library tests; keyboard operation, visible focus, labelled controls, and error text tied to fields by `aria-describedby` are part of each component's definition of done (Q&A 49 — HEC requires 508 conformance and uses Storybook; full pass on every rule is not required, but the interactive path must work without a mouse).

What "working" looks like at the end of P1. A district administrator with no command-line access creates the SWT policy above through the UI; a cooperator's `GET /cwms-data/timeseries?name=…&office=SWT` returns data older than 72 hours and nothing newer; an operator's identical request returns everything; with the feature flag off, both requests behave exactly as before the change. That scenario is the acceptance test, and it is also the first walkthrough in the training materials.

B.4 A second requirement, in brief: show new users who have no privileges yet

With `cwms.dataapi.access.openid.create_users=true` — set in both compose files — a person's first Keycloak login creates an `at_sec_cwms_users` row with no `at_sec_users` membership. Those are the users TE1 wants surfaced, and today they are invisible: `UserDao.getAll` deliberately restricts its result to users who already have a group membership. The change is one query parameter and one join: `GET /users?office=&unassigned=true` implemented as `at_sec_cwms_users` left-joined to `av_sec_users` on office, returning rows with no membership (or, optionally, no persona-group membership), with a matching `UsersControllerTestIT`. The management API forwards the parameter; the Users page gains an **Unassigned** filter beside the office selector and an *assign role* control on each row that calls the existing `POST /user/{user-name}/roles/{office-id}`. Registration can happen at any time, so the list is live, not a snapshot.

B.5 How the UI work progresses through the contract

Increment U1 (district scope restored, role assignment working) ships in the first month alongside the PR #1461 rebase, because it exercises only endpoints CDA already has. P1 follows once the embargo ADR is accepted, since its schema change is the one decision that cannot be reversed cheaply. U2 and R1 are small and fill review gaps. R2 and U3 complete the exhibit. Documentation — an administrator guide organized by the three tabs, and a short training deck with the SWT walkthrough — is the final Task 2 work unit, delivered in the repository's `docs/source/access-management/management/` tree where the previous effort left placeholders.